

투두리스트에 날짜도 넣고 싶은데 많이 어려울까? 내가 우 너하는 날짜를 하나 넣어서 보고싶어

투두리스트에 날짜를 추가하는 건 **전혀 어렵지 않아요!** 이미 4차 과제에서 **이중 리스트(2D List)** 구조를 잡았기 때문에, 각 항목에 날짜 데이터 방을 하나만 더 만들어주면 끝납니다. [PDF](#)

기존 구조가 [이름, 할일, 중요도] 였다면, 여기에 날짜를 더해 [이름, 날짜, 할일, 중요도] 형태로 확장하는 것이죠. [PDF](#)

과제 기준(이중 리스트, 예외 처리, 파일 저장 등)을 완벽하게 만족하면서, 원하는 날짜를 직접 입력하고 조회할 수 있는 핵심 코드 로직과 설명입니다. [PDF](#)

17 날짜 추가를 위한 핵심 로직 3단계

1. 데이터 구조 확장 (이중 리스트)

사용자에게 날짜를 입력받아(`input()`) 기존 이중 리스트에 함께 `append()` 합니다. [PDF](#)

Python

```
# 예시: [이름, 날짜, 할일, 중요도] 구조
todo_data = [
    ['홍길동', '2026-06-12', '파이썬 과제하기', '높음'],
    ['이순신', '2026-06-13', '운동하기', '보통']
]
```

2. 이중 순회 출력 (날짜 포함)

중첩 for문을 사용할 때 날짜 칸(`row[1]`)을 추가해서 표 형태로 깔끔하게 출력합니다. [PDF](#)

Python

```
print("-" * 50)
print(f"{<이름>:<6}&#123; | &#123;날짜'&#123; | &#123;할 일'&#123; | &#123;중요도'&#123;")
print("-" * 50)
```

+ Gemini에게 물어보기

Flash



```
for row in todo_data:
    # row[0]=이름, row[1]=날짜, row[2]=할일, row[3]=중요도
    print(f"{row[0]:<6} | {row[1]:<12} | {row[2]:<15} | {row[3]:<5}")
print("-" * 50)
```

3. 특정 날짜 필터링 조회 기능

"원하는 날짜를 하나 넣어서 보고 싶다"고 하셨으니, 조회할 날짜를 입력받아 일치하는 데이터만 쓱쓱 뽑아주는 조건문(if)을 추가하면 됩니다. [PDF](#)

Python

```
search_date = input("조회하고 싶은 날짜를 입력하세요 (예: 2026-06-12): ")

print(f"\n [{search_date}]의 할 일 목록")
for row in todo_data:
    if row[1] == search_date: # 입력한 날짜와 리스트의 날짜가 일치할 때만 출력
        print(f"- {row[0]}님: {row[2]} ({row[3]})")
```

🔧 4차 과제용 `main.py` 실제 적용 예시 코드

과제 필수 조건인 try-except(2곳 이상)과 파일 저장(with open)까지 날짜 데이터에 맞춰 수정한 완성형 예시입니다. 이 구조를 바탕으로 직접 코드를 타이핑해 보세요. [PDF +1](#)

Python

```
# 전역 변수로 사용할 이중 리스트 선언
todo_data = []

while True:
    print("\n=== ✨ 행운의 투두리스트 시스템 V3.0 ===")
    print("1. 할 일 등록 (날짜 포함)")
    print("2. 전체 목록 조회")
    print("3. 특정 날짜로 검색")
    print("4. 파일에 저장하기")
    print("5. 프로그램 종료")

    # [예외 처리 1] 메뉴 입력 시 숫자 변환 오류 방지
    try:
        choice = int(input("메뉴를 선택하세요: "))
    except ValueError:
```

+ Gemini에게 물어보기

Flash



작성해주신 코드를 면밀히 분석해보니, 지난번보다 데이터 구조의 일관성을 맞추려고 많이 고민하신 흔적이 보입니다! 4차 과제의 핵심 요구사항인 이중 리스트 구조, 예외 처리, 파일 저장 문법도 전체적으로 잘 녹아있습니다.

다만, 데이터를 다루는 과정에서 인덱스 번호(방 번호)가 서로 엇갈려 출력 예러가 나거나 프로그램이 강제 종료되는 치명적인 논리적 버그들이 몇 군데 발견되었습니다.

과제 제출 시 감점이 없도록 문제가 되는 원인과 최종 수정 코드를 알기 쉽게 정리해 드릴게요.

🔍 치명적인 오류 및 감점 요인 분석

1. 이중 리스트 저장 순서와 인덱스(row[...])의 불일치

`add_task()` 에서 데이터를 넣을 때 순서가 다음과 같습니다.

Python

```
# [0]이름, [1]날짜, [2]할일, [3]카테고리, [4]중요도
todo_list.append([user_name, task_date, t_input, c_input, p_input])
```

하지만 `view_task()` 에서 출력하거나 비교할 때는 예전 번호 그대로 가져다 쓰셨습니다.

- 전체 목록 조회 (`choice == '1'`): `row[0]` (이름), `row[1]` (날짜), `row[2]` (할일), `row[3]` (카테고리)만 출력되어 중요도(`row[4]`)가 누락됩니다.
- 중요도 별 조회 (`choice == '2'`): `row[3]` (카테고리)를 추출해 중요도 숫자인 `select_p` 와 비교하고 있어 아무것도 매칭되지 않습니다. (`row[4] == select_p` 가 되어야 합니다.)
- 특정 날짜 별 조회 (`choice == '4'`): `row[0]` (이름)을 입력받은 날짜와 비교하고 있어 "해당 날짜에 등록된 할 일이 없습니다"라는 메시지만 무한히 뜨게 됩니다.

2. 파일 저장 시 인덱스 범위 초과 에러 (`save_to_file`)

Python

```
line = str(row[0]) + "," + ... + str(row[4]) + "\n"
```

+ Gemini에게 물어보기

Flash



현재 리스트에는 5개의 데이터([0] ~ [4])가 들어있는데, 저장할 때는 정성적으로 row[4] 까지 명시했습니다. 위의 view_task 인덱스를 고쳐주면 이 부분은 자연스럽게 잘 맞아떨어집니다. 다만, except ValueError: 로 잡아두셨는데 파일 저장 실패는 주로 파일 입출력 권한이나 시스템 문제로 발생하므로 범용적인 except Exception: 을 쓰는 것이 가이드라인(안정성 확보)에 더 유리합니다.

3. main() 메뉴판 안내와 입력 번호의 한계

main() 메뉴판에서는 1년부터 5번 까지 선택할 수 있게 출력해두고, 잘못된 입력을 걸러내는 마지막 else: 구문에서는 "1년부터 4번 사이의 메뉴를 입력해 주세요." 라고 잘못 출력하고 있습니다.

🔧 최종 완벽 수정 코드 (main.py)

인덱스 매칭과 오타를 전부 수정한 최종본입니다. 이 코드는 날짜 검색, 카테고리 검색, 중요도 검색, 전체 출력이 유기적으로 완벽하게 작동합니다.

Python

```
todo_list = []

def add_task():
    global todo_list
    print("\n----[입력 메뉴] 할 일 등록----")

    try:
        user_name = input("작성자 이름을 입력하세요: ")
        task_date = input("날짜 입력 (예: 2026-06-12): ")
        t_input = input("투두리스트 입력: ")

        while True:
            c_input = input("목록(공부/취미/자기개발/기타): ")
            if c_input in ['공부', '취미', '자기개발', '기타']:
                break
            else:
                print("지정된 카테고리 중 하나만 입력해주세요.")

        while True:
            p_input = int(input("중요도(1-3): "))
            if 1 <= p_input <= 3:
                break
            else:
                print("중요도는 1, 2, 3 중 하나만 입력해주세요")

        # 인덱스 순서 규칙: [0]이름, [1]날짜, [2]할일, [3]카테고리, [4]중요도
        todo_list.append([user_name, task_date, t_input, c_input, p_input])
        print(f"👍 {t_input}가 성공적으로 저장되었습니다.")
```

+ Gemini에게 물어보기

Flash



@workspace /explain 식이 필요합니다.

Optimized tool selection

I will inspect the current code and point out what the selected expression means in this file, then I can help fix or clarify it if needed.

Reviewed main.py and clarified code explanation

선택하신 `ex` 는 `except` 의 앞부분입니다. 현재 `main.py` 에서 이 구조는 다음 뜻이에요:

- `try:` 안에 있는 코드가 실행됩니다.
- 그 과정에서 `ValueError` 같은 오류가 나면,
- `except ValueError:` 아래의 코드가 실행됩니다.

즉, 사용자가 숫자 대신 문자를 입력했을 때 프로그램이 강제 종료되지 않고, 대신 "숫자로만 입력해야 합니다." 같은 안내 문구를 보여줍니다.

예를 들어 이 부분에서:

- `int(input("중요도(1-3): "))`
- `int(input('조회할 중요도(1-3): '))`

사용자가 `abc` 를 입력하면 `int("abc")` 가
`ValueError` 를 일으키고, 그때 `except` 가 잡아
서 처리합니다.

